Universidade do Minho
Licenciatura em Engenharia Informática

RELATÓRIO

LABORATÓRIOS DE INFORMÁTICA III

FASE 2

Trabalho realizado por:

Ana Patrícia da Silva Machado (*a111661*)
Alexandra Cerqueira da Luz das Neves (*a110903*)
Carolina Francisca Vieira Dias (*a111179*)
GRUPO 12

Janeiro, 2026

ÍNDICE

1. INTRODUÇÃO.....1

2. SISTEMA.....2

2.1 Estrutura do Projeto.....2

2.2 Arquitetura Modular.....3

2.3 Encapsulamento.....3

2.4 Executáveis.....3

2.5 Compilação.....4

2.6 Gestão de Memória.....4

2.7 Bibliotecas Utilizadas.....4

3. DISCUSSÃO.....5

3.1 Técnicas de Encapsulamento.....5

3.2 Escolha de Estruturas de Dados.....6

3.3 Análise de Desempenho.....6

3.4 Justificação dos Resultados.....8

3.5 Melhorias Futuras.....8

4. CONCLUSÃO.....9

1. INTRODUÇÃO

O presente relatório descreve o desenvolvimento da Fase 2 do trabalho prático da Unidade Curricular de Laboratórios de Informática III, da Universidade do Minho. Este projeto tem como objetivo central a implementação de um sistema robusto em linguagem C, capaz de processar e analisar ficheiros de dados no formato CSV, simulando a gestão de um sistema de informação relacionado com aeroportos, aeronaves, voos, passageiros e reservas.

A segunda fase foi marcada por uma expansão significativa do trabalho, tanto ao nível do volume de dados manipulados como das funcionalidades disponibilizadas. Foi alterada uma querie já existente (Q1) e foram implementadas três novas queries (Q4, Q5 e Q6), envolvendo operações mais complexas de filtragem, agregação e cálculo de métricas, bem como um modo de execução interativo que permite ao utilizador executar queries diretamente através do terminal.

A arquitetura do programa segue uma abordagem modular, com separação clara entre ficheiros de interface (*.h*) e implementação (*.c*), cada um dedicado a responsabilidades específicas. Esta organização promove a manutenção e reutilização de código, sendo apoiada pelo princípio de encapsulamento, que garante a proteção das estruturas de dados internas contra acessos externos indevidos. Nesta fase, foi dado particular ênfase ao encapsulamento, assegurando que nenhuma estrutura de dados da biblioteca GLib é exposta nos ficheiros de interface públicos.

A informação extraída dos ficheiros CSV é armazenada em estruturas de dados dinâmicas, permitindo otimizar o acesso à informação e garantir um bom desempenho em operações de leitura intensiva. Para reforçar a integridade do sistema, foram implementados mecanismos de validação sintática e lógica em todas as entidades, com registo das entradas inválidas em ficheiros de erro próprios a cada entidade.

Procedeu-se igualmente à análise do desempenho do programa, avaliando o impacto das escolhas de implementação e das estruturas de dados selecionadas. Para tal, foi utilizado o executável *programa-testes*, com execução repetida das queries e recolha dos tempos médios de execução em diferentes ambientes de hardware. Os resultados obtidos são discutidos neste relatório, tendo em conta a complexidade dos algoritmos desenvolvidos. Recorreu-se à ferramenta Valgrind para garantir a ausência de fugas de memória.

O controlo de versões foi assegurado através do Git e da plataforma GitHub, possibilitando o trabalho colaborativo entre os membros do grupo e o rastreio contínuo das alterações efetuadas.

Este relatório apresenta a arquitetura do sistema, as principais decisões técnicas adotadas, uma análise crítica do desempenho obtido e as considerações finais sobre o estado atual do projeto.

2. SISTEMA

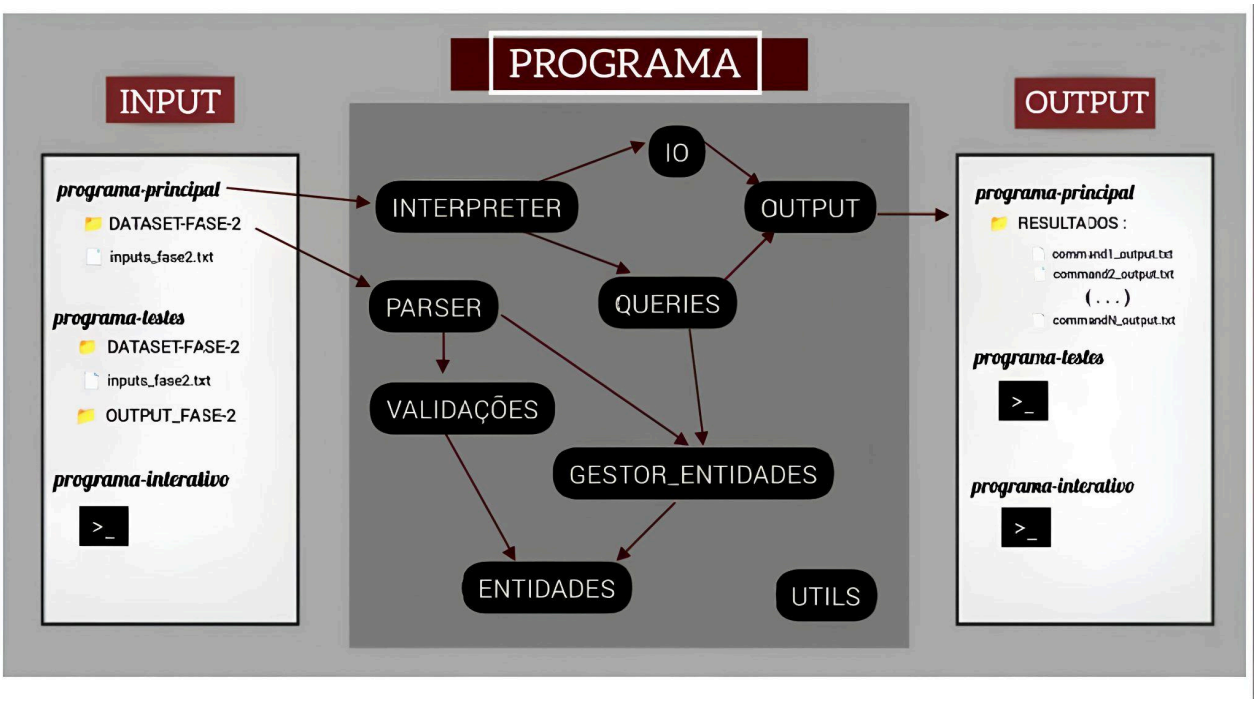

Fig 1 - Diagrama da Arquitetura do Sistema

O programa foi desenvolvido em linguagem C, seguindo uma arquitetura modular que promove a separação clara de responsabilidades, a reutilização de código e a escalabilidade do sistema. Na Fase 2, o sistema foi significativamente expandido, passando a suportar a leitura, validação e processamento de cinco ficheiros CSV (*airports.csv*, *aircrafts.csv*, *flights.csv*, *passengers.csv* e *reservations.csv*), permitindo uma gestão completa e integrada da informação.

A informação carregada é armazenada dinamicamente em memória através de estruturas de dados eficientes, otimizadas para acesso e pesquisa em tempo constante, fundamental para o desempenho das queries executadas.

2.1 Estrutura do Projeto

A organização do projeto encontra-se no diretório **trabalho-pratico/,** estruturado da seguinte forma:

- **include/**: ficheiros de cabeçalho (*.h*), que definem as interfaces públicas e declarações de tipos opacos;
- **src/**: implementação dos módulos (*.c*), organizados por entidades, gestores, queries, parsers, validações e I/O;
- **programas/**: ficheiros fonte dos três executáveis principais;
- **resultados/**: ficheiros de saída gerados após a execução das queries;
- **Makefile**: responsável pela compilação e geração dos executáveis.

2.2 Arquitetura Modular

O sistema está subdividido nas seguintes áreas funcionais:

- **Entidades**: representação das estruturas de dados do domínio (aeroportos, aeronaves, voos, passageiros e reservas);
- **Gestores**: módulos responsáveis pelo armazenamento, acesso e manipulação das entidades;
- **Parsers**: leitura e processamento dos ficheiros CSV;
- **Validações**: verificação da integridade sintática e lógica dos registos;
- **Interpreter**: módulo responsável por interpretar os comandos do ficheiro de input e invocar a query correspondente;
- **Queries (Q1 a Q6)**: lógica de processamento para cada interrogação;
- **I/O**: leitura de comandos e escrita de resultados;
- **Utils**: funções auxiliares de uso geral.

Esta separação garante que cada módulo tem uma responsabilidade específica, favorecendo a manutenção e evolução futura do sistema.

2.3 Encapsulamento

O projeto segue rigorosamente o princípio de encapsulamento. Todas as estruturas de dados são declaradas como tipos opacos nos ficheiros de interface (*.h*), sendo a sua definição concreta mantida exclusivamente nos ficheiros de implementação (*.c*). Esta abordagem garante que:

- Os detalhes internos de cada módulo permanecem protegidos de acessos externos;
- As alterações à implementação não afetam os módulos dependentes;
- A interface pública de cada módulo é clara e bem definida.

Adicionalmente, foi implementado um tipo opaco *StringPool* para gestão centralizada de strings, permitindo otimização de memória através de *string interning,* sem expor detalhes de implementação.

2.4 Executáveis

O desenvolvimento originou três executáveis distintos:

- **programa-principal**: executa queries com base num ficheiro de comandos, gerando os resultados na pasta **resultados/**;
- **programa-interativo**: disponibiliza uma interface em terminal que permite ao utilizador executar queries de forma interativa, com validação de comandos e argumentos;
- **programa-testes**: compara os resultados obtidos com os esperados, apresentando métricas de desempenho e validação.

2.5 Compilação

A compilação é automatizada através do Makefile, que assegura:

- Compilação individual dos módulos com geração de ficheiros objeto (*.o*);
- Ligação e geração dos executáveis finais;
- Comandos de limpeza do projeto (**make clean**);
- Compatibilidade com o ambiente de validação da plataforma de testes.

2.6 Gestão de Memória

Para assegurar robustez e fiabilidade:

- Todos os registos inválidos são descartados durante o parsing;
- Foram aplicadas boas práticas de alocação dinâmica;
- A ferramenta Valgrind foi utilizada para detetar e eliminar fugas de memória;
- Todas as estruturas são libertadas no término da execução, garantindo 0 bytes de memória perdida.

2.7 Bibliotecas Utilizadas

Além da biblioteca GLib para estruturas de dados, foram utilizadas bibliotecas padrão de C:

- *stdio.h*, *stdlib.h*: gestão de memória e I/O;
- *string.h*, *ctype.h*: manipulação de strings;
- *time.h*: tratamento de datas;
- *math.h*: operações matemáticas (arredondamentos).

3. DISCUSSÃO

O desenvolvimento da Fase 2 introduziu novos desafios técnicos, tanto ao nível da estrutura do programa como da eficiência do processamento. Para além da execução automática de queries a partir de um ficheiro de comandos, foi implementado um modo interativo que permite ao utilizador executar queries em tempo real através do terminal, exigindo validação rigorosa de todos os inputs para evitar comportamentos inesperados.

3.1 Técnicas de Encapsulamento

O projeto segue rigorosamente o princípio de encapsulamento, sendo esta uma das principais melhorias em relação à Fase 1. Todas as estruturas de dados são declaradas como **tipos opacos** nos ficheiros de interface (*.h*), sendo a sua definição concreta mantida exclusivamente nos ficheiros de implementação (*.c*).

As principais técnicas de encapsulamento aplicadas foram:

- **Tipos opacos:** todas as estruturas (Voo, Aeroporto, Aeronave, Passageiro, Reserva, Gestor*) são declaradas nos headers apenas como typedef struct _Nome Nome, escondendo os campos internos;
- **Getters e Setters:** o acesso aos campos das estruturas é feito exclusivamente através de funções dedicadas, nunca por acesso direto;
- ***StringPool* opaco:** foi implementado um tipo opaco para gestão centralizada de strings (string interning), permitindo otimização de memória sem expor detalhes de implementação;
- **Gestores encapsulados:** cada entidade possui um gestor dedicado que encapsula as operações de armazenamento e acesso, expondo apenas funções de alto nível;
- **Funções static:** funções auxiliares são declaradas como static para garantir visibilidade apenas interna ao módulo;
- **Zero estruturas expostas:** nenhuma estrutura de dados da biblioteca GLib é exposta nas interfaces públicas.

Esta abordagem garante que os detalhes internos de cada módulo permanecem protegidos, permitindo alterações à implementação sem afetar os módulos dependentes.

3.2 Escolha de Estruturas de Dados

A escolha das estruturas de dados foi orientada pela necessidade de garantir acesso eficiente aos dados e bom desempenho nas queries. As principais estruturas utilizadas foram:

- **Tabelas de dispersão (Hash Tables):** utilizadas nos gestores para armazenar entidades indexadas por identificador único, permitindo acesso em tempo médio O(1);
- **Array ordenado por data:** os voos são mantidos num array ordenado por data de partida, permitindo pesquisa binária O(log n) para queries temporais (Q3);
- ***StringPool* (String Interning):** mecanismo que evita a duplicação de strings idênticas em memória, reduzindo significativamente o consumo de memória;
- **Caches pré-computados:** estruturas auxiliares calculadas durante o parsing para acelerar queries frequentes.

As otimizações específicas para cada query foram:

- **Q1:** contagens de partidas e chegadas por aeroporto pré-calculadas durante o parsing;
- **Q2:** contagens de voos por aeronave armazenadas em tabela auxiliar, evitando percorrer todos os voos;
- **Q3:** array de voos ordenado por data com pesquisa binária para encontrar intervalos temporais;
- **Q4:** cache do Top 10 semanal de passageiros com maior gasto em reservas;
- **Q5:** métricas de atraso por companhia aérea calculadas incrementalmente durante o parsing;
- **Q6:** cache de destinos por nacionalidade pré-computado.

Estas otimizações permitiram reduzir o tempo de execução de aproximadamente 5 minutos para cerca de 45 segundos no dataset de maior dimensão, trocando memória por velocidade de processamento.

3.3 Análise de Desempenho

A análise de desempenho foi realizada com o executável *programa-testes* nos equipamentos dos três elementos do grupo, repetindo cada teste 3 vezes para eliminar variações causadas pelo escalonamento do sistema operativo.

Tabela 1 - Especificações dos equipamentos de teste

Elemento	Máquina	CPU	RAM	Sistema
a111661	DELL	Intel Core i5-8365U	16 GB	Ubuntu 24.04
a110903	HUAWEI	AMD Ryzen 5 3500U	8 GB	Ubuntu 24.04
a111179	LENOVO	AMD Ryzen 5 7530U	16 GB	Ubuntu 24.04

Tabela 2 - Tempos de execução no Dataset Regular (Fase 1)

Elemento	Q1	Q2	Q3	Q4	Q5	Q6	Total
a111661	62 ms	61 ms	112 ms	74 ms	75 ms	78 ms	12.2 s
a110903	74 ms	75 ms	136 ms	81 ms	83 ms	86 ms	26.9 s
a111179	51 ms	51 ms	90 ms	50 ms	51 ms	52 ms	4.5 s

Tabela 3 - Tempos de execução no Dataset Large (Fase 2)

Elemento	Q1	Q2	Q3	Q4	Q5	Q6	Total
a111661	726 ms	722 ms	914 ms	725 ms	726 ms	811 ms	80 s
a110903	1197 ms	1191 ms	1495 ms	1157 ms	1178 ms	1186 ms	370,3 s
a111179	589 ms	588 ms	784 ms	589 ms	585 ms	597 ms	45 s

Tabela 4 - Consumo de memória

Dataset	Memória (MB)	Limite
Regular (Fase 1)	~289 MB	4096 MB
Large (Fase 2)	~2818 MB	4096 MB

3.4 Justificação dos Resultados

Os resultados obtidos refletem as otimizações implementadas e as características do hardware utilizado.

Tempo de execução: As queries Q1, Q2 e Q5 apresentam tempos muito reduzidos devido às estruturas pré-computadas durante o *parsing*. A Q3 beneficia do array ordenado com pesquisa binária. A Q4 e Q6 são as mais exigentes devido à necessidade de agregar dados de múltiplas entidades, mas os *caches* implementados garantem desempenho adequado.

Consumo de memória: O consumo de aproximadamente 2.8 GB para o dataset large é justificado pelo volume de dados processados (5.6 milhões de voos, 4 milhões de reservas, 2 milhões de passageiros) e pelas estruturas de cache que aceleram as queries. A utilização de *StringPool* (string interning) evita duplicação de strings idênticas, reduzindo significativamente o consumo que seria necessário sem esta otimização. O valor obtido encontra-se dentro do limite de 4 GB estabelecido pela plataforma.

Variações entre máquinas: As diferenças de tempo entre equipamentos são atribuíveis principalmente ao tipo de armazenamento (SSD vs HDD) e à quantidade de RAM disponível. Equipamentos com SSD apresentam tempos de carregamento significativamente inferiores.

Validação na plataforma: Os testes na plataforma de avaliação automática confirmam um desempenho consistente, com todas as queries a produzir resultados corretos (100% de precisão) dentro dos limites de tempo estabelecidos.

3.5 Melhorias Futuras

Identificamos possíveis melhorias para trabalho futuro:

- Implementação de cache de resultados para queries repetidas no modo interativo;
- Paralelização do carregamento dos ficheiros CSV para reduzir tempo de inicialização;
- Otimização adicional do consumo de memória através de compressão de dados pouco acedidos;
- Implementação de índices secundários para acelerar queries com filtros complexos.

4. CONCLUSÃO

O desenvolvimento deste projeto permitiu consolidar e aplicar conhecimentos fundamentais de programação em linguagem C, estruturas de dados dinâmicas, gestão de memória e princípios de engenharia de software, com particular ênfase em encapsulamento e modularidade.

A Fase 2 introduziu desafios significativos face à fase anterior, nomeadamente o aumento substancial do volume de dados processados, passando de milhares para milhões de registos, a implementação de três novas queries com operações de agregação e filtragem mais complexas, e a criação de um modo de execução interativo com validação rigorosa de inputs. Estes desafios foram superados através de uma arquitetura modular assente em tipos opacos, estruturas de dados otimizadas para acesso eficiente e caches pré-computados durante a fase de parsing, permitindo reduzir o tempo de execução de aproximadamente cinco minutos para cerca de quarenta e cinco segundos no dataset de maior dimensão.

Entre os aspetos mais bem conseguidos, destaca-se o encapsulamento rigoroso do código, garantindo que nenhuma estrutura de dados interna é exposta nas interfaces públicas, bem como a gestão de memória validada sistematicamente com a ferramenta Valgrind, que confirmou a ausência de fugas de memória. A modularidade adotada facilitou a adição de novas funcionalidades sem comprometer o código existente.

Não obstante, o desenvolvimento não foi isento de dificuldades. A adaptação ao ambiente de testes automático revelou discrepâncias face à execução local, exigindo revisões na estrutura do projeto e no formato dos outputs. O balanceamento entre consumo de memória e velocidade de execução constituiu igualmente um desafio, tendo-se optado por privilegiar o desempenho temporal dentro dos limites de memória estabelecidos.

Como trabalho futuro, identificam-se possíveis otimizações, designadamente a implementação de cache de resultados para queries repetidas no modo interativo, a paralelização do carregamento dos ficheiros CSV e a criação de índices secundários para acelerar operações de filtragem mais complexas.

A utilização de Git e GitHub revelou-se essencial para a coordenação do trabalho em equipa, permitindo o controlo de versões e a integração contínua das contribuições de cada elemento. Em suma, o projeto cumpriu os objetivos propostos, proporcionando uma experiência prática completa que abrangeu não apenas a implementação técnica, mas também a organização colaborativa e a aplicação de metodologias de desenvolvimento profissional.